CSC321: Embedded Systems

Practical Assessment

Input and Output Devices Programming

Platform: ESP32 with MicroPython
Total Marks: 100

Student Name: _____

Student ID: _____

General Instructions

1. Complete all five exercises using MicroPython on one ESP32 development board.
2. Use the GPIO assignments specified in the hardware pin map throughout the practical.
3. Test each program on the physical hardware.
4. Use meaningful variable names, functions, classes and comments where appropriate.
5. Where interrupts are required, interrupt handlers must be short and must not contain blocking delays or printing operations.
6. Demonstrate the required behaviour to the instructor and submit the MicroPython source files.
7. Ensure all devices share a common ground. Observe the electrical safety instructions.

Hardware Pin Map

Use the following assignments so that the same wired ESP32 board can be used for all exercises.

Wiring Safety

- The HC-SR04 ECHO pin can output 5 V, whereas ESP32 GPIO inputs operate at 3.3 V. Use a voltage divider with 1 kΩ between ECHO and GPIO 33 and 2 kΩ between GPIO 33 and GND.
- Connect the grounds of all devices and external power supplies together.
- Keep each NeoPixel colour channel at 40 or below unless the ring has a suitable external 5 V supply.
- Use suitable current-limiting resistors for discrete LEDs.
- Verify that the selected ESP32 board supports the GPIO assignments before wiring.

1. **Input-to-Output Verification Tests** [15]

   Run each test below, press `Ctrl+C` to stop it, and record your observations.

   Each test demonstrates how one input device can control an output device before building more complex systems.

Device	Direction	GPIO	Notes
LED	Output	19	External LED or board-specific LED
RGB LED (common cathode)	Output	R: 21, G: 22, B: 23	Drive with PWM
Traffic light module	Output	Red: 18, Yellow: 17, Green: 16	Three-LED module
Passive buzzer	Output	25	Drive with PWM
NeoPixel ring (WS2812B)	Output	26	Use `NUM_LEDS = 8` for an 8-LED ring
Push button 1	Input	15	Connect to GND; internal pull-up
Push button 2	Input	4	Connect to GND; internal pull-up
PIR motion sensor	Input	27	HIGH indicates motion; allow warm-up
HC-SR04 ultrasonic sensor	TRIG output, ECHO input	TRIG: 32, ECHO: 33	Powered from 5 V; level-shift ECHO

(a) **Push Button and LED** [5]

The LED must remain ON only while button 1 is pressed.

```
from machine import Pin
from time import sleep_ms

button = Pin(15, Pin.IN, Pin.PULL_UP)
led = Pin(19, Pin.OUT)

try:
    while True:
        led.value(not button.value())
        sleep_ms(20)
except KeyboardInterrupt:
    led.off()
```

Task: Run the program and explain why pressing the button causes the input to read 0.

(b) **PIR Sensor and LED** [5]

After the PIR sensor warms up, the LED should turn ON when motion is detected.

```
from machine import Pin
from time import sleep, sleep_ms

pir = Pin(27, Pin.IN)
led = Pin(19, Pin.OUT)

print("PIR warming up...")
sleep(30)
print("PIR ready")

try:
    while True:
        led.value(pir.value())
        sleep_ms(50)
except KeyboardInterrupt:
    led.off()
```

Task: Observe the LED when motion is detected and when no motion is detected.

(c) **Ultrasonic Sensor and Traffic Light** [5]

Place an object at approximately 10 cm and then 50 cm from the ultrasonic sensor.

The red LED must turn ON below 20 cm, while the green LED must turn ON at 20 cm or greater.

```
from machine import Pin, time_pulse_us
from time import sleep_ms, sleep_us

trig = Pin(32, Pin.OUT)
echo = Pin(33, Pin.IN)

red = Pin(18, Pin.OUT)
green = Pin(16, Pin.OUT)

def read_cm():
    trig.value(0)
    sleep_us(2)
    trig.value(1)
    sleep_us(10)
    trig.value(0)

    pulse = time_pulse_us(echo, 1, 30000)

    if pulse < 0:
        return -1

    return (pulse * 0.0343) / 2

try:
    while True:
        distance = read_cm()
        print("Distance:", distance)

        if distance >= 0:
            red.value(distance < 20)
            green.value(distance >= 20)
        else:
            red.off()
            green.off()

        sleep_ms(200)

except KeyboardInterrupt:
    red.off()
    green.off()
```

Task: Record the distance readings and the corresponding traffic light outputs.

2. **Interrupt-Driven RGB Colour Selector** [20]

Develop an RGB lamp controller in which two push buttons change the colour and brightness of an RGB LED using interrupts.

The main loop must not poll the button pins.

(a) **PWM Colour Output** [5]

Create PWM objects for the RGB pins at 1000 Hz.

Write a function:

```
set_color(color, level)
```

where:

- `color` is an RGB tuple containing values from 0 to 255.
- `level` is a brightness percentage from 0 to 100.

Convert each channel to a 16-bit PWM duty value between 0 and 65535.

(b) **Interrupt Handlers** [5]

Attach falling-edge interrupts to both buttons.

- Each handler must be short and must only set a flag for the main loop.
- Do not use `sleep()` or `print()` inside an interrupt handler.
- Ignore presses occurring within 200 ms of the previous accepted press on that button.
- Use `ticks_ms()` and `ticks_diff()` for debouncing.

(c) **Main Loop Behaviour** [6]

Implement the following behaviour:

1. Button 1 cycles through red, green, blue, yellow, cyan and magenta, then returns to red.
2. Button 2 cycles brightness through 25%, 50%, 75% and 100%, then returns to 25%.
3. Each colour change produces a 100 ms beep at 1000 Hz.
4. Each brightness change produces a 100 ms beep at 1500 Hz.
5. Print the current colour and brightness after every change.

Example output:

```
Colour: cyan, Brightness: 75%
```

(d) **Safe Shutdown** [4]

When `Ctrl+C` is pressed:

- Disable both button interrupts.
- Turn OFF the RGB LED.
- Silence the buzzer.
- Call `deinit()` on all PWM objects.

3. **PIR Security Alarm State Machine** [20]

Develop a room security alarm using a non-blocking finite state machine.

The system must respond to button 1 within 100 ms in every state.

Use `ticks_ms()` and `ticks_diff()` instead of long blocking delays.

State	Traffic Light	Buzzer	NeoPixel	Transition
DISARMED	All OFF	Silent	OFF	Button 1 → ARMING
ARMING	Yellow blinks at 1 Hz	One short beep per second	OFF	After 10 s → ARMED; button 1 → DISARMED
ARMED	Green	Silent	One dim green LED	Motion → ALARM; button 1 → DISARMED
ALARM	Red	880 Hz and 660 Hz alternating every 250 ms	All LEDs flash red at 2 Hz	After 30 s → ARMED; button 1 → DISARMED

(a) **Input Configuration** [4]

Configure button 1 and the PIR sensor.

- Use a debounced falling-edge interrupt for the button.
- Use a rising-edge interrupt for the PIR.
- Interrupt handlers must only set flags.

(b) **Output Helper Functions** [5]

Create separate functions or classes for:

- Traffic light
- Buzzer
- NeoPixel ring

Ensure that only one traffic light LED can be ON at any time.

(c) **State Machine** [7]

Implement all states and transitions shown in the table.

- Ignore PIR events outside the ARMED state.
- Clear stale motion flags when entering ARMED.
- Use non-blocking timing.
- Permit button 1 to disarm the system during ARMING, ARMED and ALARM.

(d) **Event Log and Shutdown** [4]

Print every state transition together with the elapsed time since the program started.

Example:

```
[42.5 s] ARMED -> ALARM
```

Count the number of alarms and print the total whenever the system enters ALARM.

On `Ctrl+C`, turn OFF all outputs and print the final alarm count.

4. **Ultrasonic Parking Assistant** [20]

Develop a parking assistant that indicates the distance to an obstacle using the traffic light, buzzer and NeoPixel ring.

Buzzer timing must not block sensor readings.

Zone	Distance (cm)	Traffic Light	Buzzer
CLEAR	$d > 100$	Green	Silent
CAUTION	$50 < d \leq 100$	Yellow	80 ms beep every 1000 ms
NEAR	$20 < d \leq 50$	Yellow	80 ms beep every 300 ms
STOP	$d \leq 20$	Red	Continuous tone

(a) **Reliable Distance Measurement** [5]

- Take one HC-SR04 reading every 60 ms.
- Store the last five valid readings.
- Use the median of the stored readings as the measured distance d.
- Handle timeouts and out-of-range readings without crashing the program.

(b) **Zone Output** [5]

Use the table to control the traffic light and buzzer according to the measured distance.

Implement all buzzer patterns without blocking the main loop.

(c) **NeoPixel Distance Bar** [6]

Light the LEDs sequentially, starting at index 0.

Calculate the number of illuminated LEDs using:

$$\text{lit} = \text{round}\left(\text{NUM_LEDS} \times \frac{100 - d}{95}\right)$$

Limit the result to the interval:

$$0 \leq \text{lit} \leq \text{NUM_LEDS}$$

Use the following colours:

- First third: Green
- Middle third: Yellow
- Final third: Red

All LEDs must be OFF at 100 cm or greater, and all LEDs must be ON at 5 cm or less.

(d) **Calibration, Fault Handling and Testing** [4]

- Button 1 stores the current distance as a reference offset.
- Button 2 clears the offset.
- Subtract the offset from subsequent measurements when displaying calibrated distance.
- After five consecutive failed readings, print `Sensor fault`, fill the ring with dim blue and silence the buzzer.
- Resume normal operation after the next valid reading.

Record the actual and measured distances.

Actual (cm)	**Measured (cm)**	**Error (cm)**
10		
30		
60		
120		

For this test, use:

$$\text{Error} = \text{Measured distance} - \text{Actual distance}$$

5. **Smart Pedestrian Crossing System** [25]

Develop a pedestrian crossing controller using all input and output devices connected to the ESP32.

Safety requirement: Vehicles and pedestrians must never receive a green signal at the same time.

Device Roles

Device	Role
Traffic light	Vehicle signal
RGB LED	Pedestrian signal: red = wait, green = walk
Button 1 or PIR rising edge	Pedestrian crossing request
Button 2	Emergency all-stop toggle
HC-SR04	Vehicle approaching when distance ≤ 50 cm
GPIO 19 LED	ON while a crossing request is pending
Buzzer	Audio guidance for pedestrians
NeoPixel ring	Walk countdown and emergency indicator

Required States

State	Vehicle	Pedestrian	Buzzer	Ring	Duration / Next State
VEHICLE_GREEN	Green	Red	Silent	OFF	Minimum 10 s; see controller rules
VEHICLE_YELLOW	Yellow	Red	Silent	OFF	2 s → ALL_RED
ALL_RED	Red	Red	Silent	OFF	1 s → WALK
WALK	Red	Green	Fast beep, 5 cycles/s	Green countdown	8 s, possible 4 s extension → CLEARANCE
CLEARANCE	Red	Red blinking at 2 Hz	Slow beep, once/s	OFF	3 s → VEHICLE_GREEN
EMERGENCY	Red	Red	Silent	Blue flash at 2 Hz	Until button 2 pressed again → CLEARANCE

(a) **Device Classes** [5]

Implement the following classes:

```
1 TrafficLight
2 PedestrianSignal
3 Buzzer
4 Ring
5 DistanceSensor
```

Each class must manage its assigned pins and provide an `off()` method or an equivalent safe-state method.

(b) **Input Events** [6]

- Use debounced interrupts for both buttons.
- Use a rising-edge interrupt for the PIR.
- Interrupt handlers must only set flags.
- Read the HC-SR04 from the main loop no more frequently than once every 60 ms.
- Keep a pedestrian request pending until the WALK state begins.

(c) **Controller Logic** [8]

Implement a non-blocking state machine with the following rules:

1. With no pending request, remain in VEHICLE_GREEN.
2. With a pending request, leave VEHICLE_GREEN after the 10 s minimum has elapsed, provided no vehicle is approaching.
3. An approaching vehicle may delay the crossing, but VEHICLE_GREEN must not exceed 20 s in total when a request is pending.

4. If the PIR output remains HIGH when the initial 8 s WALK period ends, extend WALK once by 4 s.
5. Turn OFF ring LEDs sequentially as the WALK period progresses.
6. Relight the ring LEDs when the WALK extension begins.
7. Button 2 must enter EMERGENCY immediately from any state.

Measure the 10 s minimum and 20 s maximum from entry into VEHICLE_GREEN.

(d) **Safety, Shutdown and Demonstration** [6]

Implement a function:

```
check_safe()
```

Call it after every output change.

If the vehicle signal is green or yellow while the pedestrian signal is green:

- Print `SAFETY VIOLATION`.
- Enter the EMERGENCY state.

When changing signals, establish a safe red condition before granting a new green signal.

Print every state change.

Example:

```
Road: RED, Pedestrians: WALK (beeping)
```

On `Ctrl+C`:

- Set the vehicle signal to red.
- Set the pedestrian signal to red.
- Silence the buzzer.
- Turn OFF the NeoPixel ring.
- Disable all interrupts.

Demonstrate the following scenarios:

Scenario	Expected Result
Button pressed after 12 s of green, with no approaching vehicle	Crossing cycle starts immediately
Button pressed after 2 s of green	Green remains until the 10 s minimum; request LED stays ON until WALK
Object held at 30 cm with a request pending	Green holds, then ends at 20 s
PIR remains HIGH at the end of WALK	WALK extends once by 4 s
Button 2 pressed during WALK	Immediate EMERGENCY; second press enters CLEARANCE
Ultrasonic sensor disconnected	Program does not crash; prints a fault and treats no vehicle as approaching

Submission Requirements

Submit the following MicroPython source files:

1. `exercise1.py`

2. `exercise2.py`
3. `exercise3.py`
4. `exercise4.py`
5. `exercise5.py`

Students must demonstrate the required functionality using the ESP32 development board.

Marking Summary

Exercise	**Maximum**	**Awarded**
Exercise 1	15	
Exercise 2	20	
Exercise 3	20	
Exercise 4	20	
Exercise 5	25	
Total	**100**	

End of Practical Assessment